

OpenCode Summer Induction Task

Frontend :-

DevLore — Tech Quiz & Learning Tracker

Tech Stack

- React
 - React Router
 - localStorage
 - Fetch API
 - Open Trivia DB API
 - GitHub
-

Project Overview

Build a quiz platform that tests technical knowledge and tracks user progress over time. Every quiz attempt should be stored using localStorage so users can track their:

- Quiz history
- Strengths
- Weaknesses
- Overall improvement

The objective of this project is to help students understand how a real React application works through practical implementation.

Features to Build

1. Home Page

Display the following information:

- Total quizzes attempted
- Average score
- Best performing category
- Recent quiz history

Condition

If no quiz history exists, display:

“Start Your First Quiz”

2. Quiz Setup Page

Allow users to:

- Select quiz category
- Select difficulty level
- Start quiz

Requirement

Fetch **10 questions** from the Open Trivia DB API.

3. Quiz Page

Features

- Display one question at a time
 - Multiple choice answers
 - Timer for each question
 - Score tracking
 - Automatically move to the next question when the timer ends
-

4. Results Page

Display:

- Final score
- Number of correct answers
- Number of wrong answers
- Incorrect questions along with correct answers

Requirement

Add a button to save quiz history into localStorage.

5. Progress Dashboard

Display:

- Previous quiz sessions
- Strongest category
- Weakest category
- Score trend over time

Important

Build score bars using **CSS only**.

Do NOT use chart libraries.

localStorage Requirement

Before implementing storage, design your localStorage schema first.

Think about:

- How one quiz session will be stored
- How multiple sessions will be managed
- How category-wise statistics will be calculated

Requirement

Include the schema design in your README file.

Concepts You Will Learn

Through this project, you will learn:

- React fundamentals
 - State management
 - React Router
 - API fetching
 - localStorage
 - Timer logic
 - Conditional rendering
 - Component structure
-

Bonus Features (Optional)

You may additionally implement:

- Dark mode
 - Custom hooks
 - Leaderboard system
-

Deployment

Deploy your project using:

- Vercel
 - or
 - Netlify
-

GitHub Expectations

- Commit regularly
 - Use meaningful commit messages
 - Maintain proper project structure
 - Push changes consistently
-

Important Note

Avoid using AI tools to directly build the project.

You may use AI only for:

- Error solving
- Understanding concepts
- Learning purposes

The primary goal is to genuinely learn by building the project yourself.

BACKEND PROJECT

Inkwell — Reading Log API

Tech Stack

- Node.js + Express
or
 - Next.js API Routes
 - MongoDB
-

Project Overview

Build a backend API for storing and managing books you have read.

Users should be able to:

- Add books
- Update reading status
- Give ratings
- Track reading history

The goal of this project is to understand backend fundamentals through a real-world application instead of building a basic CRUD-only project.

Features to Build

1. Book Schema

Each book should contain:

- Title
- Author
- Genre
- Rating
- Reading Status
 - Reading
 - Finished
- Short Review

Important

Design your schema before implementation.

2. CRUD Routes

Create the following API routes:

Create Book

POST /books

Get All Books

GET /books

Get Single Book

GET /books/:id

Update Book

PATCH /books/:id

Delete Book

DELETE /books/:id

3. Filtering

Support basic filters such as:

GET /books?genre=fiction&status=finished

4. Reading Insights

Create the following route:

GET /insights

Return:

- Total books
- Average rating
- Most-read genre

You may calculate these using:

- JavaScript
or
- MongoDB queries

5. Validation & Error Handling

Handle the following cases properly:

- Missing fields
- Invalid IDs
- Invalid ratings
- Book not found

Use Proper Status Codes

- 400 — Bad Request
- 404 — Not Found
- 500 — Internal Server Error

6. Minimal Frontend

Keep the frontend minimal and use it only for:

- Backend integration
- API testing

Additional Instructions

- Understand the project requirements carefully before starting
- Refactor and improve the project wherever necessary
- Try building something genuinely useful
- Use GitHub properly
- Commit every feature separately
- Maintain clean project structure

Learning Goals

Through this project, you will learn:

- Backend fundamentals
 - REST APIs
 - CRUD operations
 - Database schema design
 - Filtering & query handling
 - Validation
 - Error handling
 - API structure
 - MongoDB integration
-

Final Note:

Deadline

June 1st, 2026

Focus on:

- Learning consistently
- Building consistently
- Understanding concepts properly

If you face any problems during development, feel free to contact your mentors. They will guide and help you whenever needed.

All The Best to Everyone